CPS style value
Pattern matching: https://en.wikibooks.org/wiki/Haskell/Continuation_passing_style#Example:_Implementing_pattern_matching
A value is represented as a **fold function** with the value.

```
-- normal data
--   A=nullary
--   B,C=n-ary
--   D=recursive
data MyValue x y = A | B x | C x y | D x (MyValue x y)

foldMyValue :: r -> (x -> r) -> (x -> y -> r) -> (x -> r -> r) -> MyValue x y -> r
foldMyValue r _ _ _ A = r
foldMyValue _ kb _ _ (B x) = kb x
foldMyValue _ _ kc _ (C x y) = kc x y
foldMyValue r kb kc kd (D x m) = kd x (foldMyValue r kb kc kd m)

-- MyValue in CPS
data M x y = M (forall r. (r -> (x -> r) -> (x -> y -> r) -> (x -> r -> r) -> r))
M x y ~ fold of MyValue x y
runM (M m) = m
M $ \r _ _ _ -> r
~ fold with A
M $ \_ kb _ _ -> kb x0
~ fold with B x0
M $ \_ _ kc _ -> kc x0 y0
~ fold with C x0 y0
M $ \r kb kc kd -> kd x0 (runM m0 r kb kc kd)
~ fold with D x0 m0
```

e.g. Maybe in CPS

```
-- Usual Maybe
data Maybe a = Just a | Nothing                -- Usual
Maybe
Just 2
Nothing

-- CPS Maybe
data CMaybe a = CMaybe { runCMaybe :: forall r. r -> (a
-> r) -> r) }

just :: a -> CMaybe a
just a = CMaybe $ \_ k -> k a                  -- ~ Just
a

nothing :: CMaybe a
nothing = CMaybe $ \r _ -> r                   -- ~
Nothing

runCMaybe (just 2) Nothing Just                -- Just 2
runCMaybe nothing Nothing Just                 -- Nothing


-- fold with Maybe
CMaybe :: (forall r. r -> (a -> r) -> r)) -> CMaybe a
maybe :: r -> (a -> r) -> Maybe a -> r
maybe 0 (*2) (Just 2)                          -- 4
```

```
maybe 0 (*2) Nothing                          -- 0
runCMaybe (just 2) 0 (*2)                     -- 4
runCMaybe nothing 0 (*2)                      -- 0
```

e.g. Either in CPS

```
-- Usual Either
data Either a b = Left a | Right b
Left 1
Right True


-- CPS Either
data CEither a b = CEither { runCEither :: forall r. (a
-> r) -> (b ->r) -> r }

left :: a -> CEither a b
left a = CEither $ \kl _ -> kl a                  -- ~
Left a

right :: b -> CEither a b
right b = CEither $ \_ kr -> kr b                 -- ~
Right b

runCEither (left 2) Left Right                    --
Left 2
runCEither (right True) Left Right                --
Right True


-- fold with Either
CEither :: (forall r. (a -> r) -> (b ->r) -> r) ->
CEither a b
either :: (a -> r) -> (b -> r) -> Either a b -> r
either (*2) not (Left 2)                          -- 4
either (*2) not (Right True)                      -- False
runCEither (left 2) (*2) not                      -- 4
runCEither (right True) (*2) not                  -- False
```